

1. Network Elements

- Let us learn the language of networks:
 - The components: nodes, links
 - Types of networks and representations
 - Features of nodes and links
- We want to be able to talk about:
 - properties to characterize structure & behaviour of networks
 - roles of networks in affecting processes occurring on network structures

Formal definitions

A **network** or **graph** G has two parts: a set of N elements, called **nodes** or **vertices**, and a set of L pairs of nodes, called **links** or **edges**. The link (i, j) joins the nodes i and j . Two nodes are **adjacent** or **connected** or **neighbours** if there is a link between them.

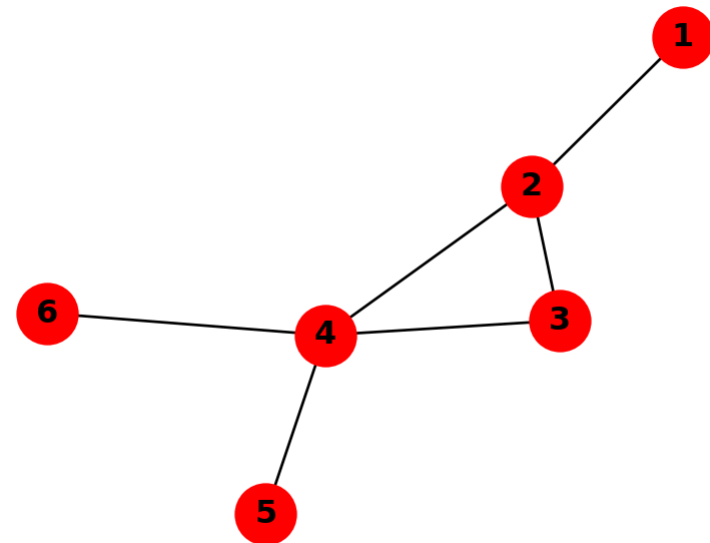
A network can be **undirected** or **directed**. A directed network is also called a **digraph**. In directed networks, links are called **directed links** and the order of the nodes in a link reflects the direction: the link (i, j) goes from the **source** node i to the **target** node j . In undirected networks, all links are bi-directional and the order of the two nodes in a link does not matter.

A network can be **unweighted** or **weighted**. In a weighted network, links have associated **weights**: the **weighted link** (i, j, w) between nodes i and j has weight w . A network can be both directed and weighted, in which case it has directed weighted links.

Python and NetworkX

```
1 import networkx as nx # always!
2
3 # Build network
4 G = nx.Graph()
5 G.add_node(1)
6 G.add_nodes_from([2,3,...])
7 G.add_edge(1,2)
8 G.add_edges_from([(2,3),(2,4),...])
9
10 # Show nodes and edges
11 print(G.nodes())
12 print(G.edges())
13
14 # Show neighbours of node 1
15 print(G.neighbors(1))
16
17 # List edges
18 for u,v in G.edges:
19     print(u, v)
```

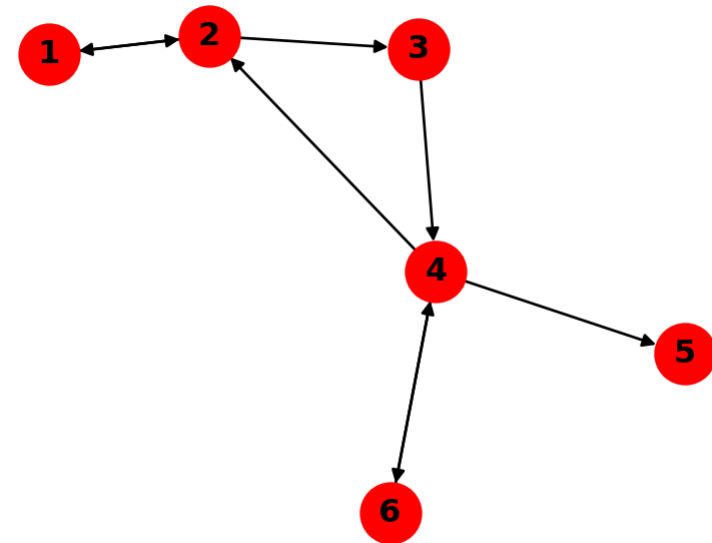
```
[1, 2, 3, 4, 5, 6]
[(1, 2), (2, 3), (2, 4), (3, 4), (4, 5), (4,
6)]
<dict_keyiterator object at 0x15c94b330>
1 2
2 3
2 4
3 4
4 5
4 6
```



Directed networks

```
1 import networkx as nx # always!
2 import matplotlib.pyplot as plt
3
4 D = nx.DiGraph()
5 D.add_edge(1,2)
6 D.add_edge(2,1)
7 ...
8 D.add_edges_from([(2,3),(3,4),...])
9
10 print(D.number_of_nodes())
11 print(D.number_of_edges())
12
13 print(D.edges())
14 print(D.successors(2)) # iterators
15 D.predecessors(2)
16 D.neighbors(2)
17
18 # List successors of node 2
19 for s in D.successors(2):
```

```
6
8
[(1, 2), (2, 1), (2, 3), (3, 4), (4, 2), (4,
5), (4, 6), (6, 4)]
<dict_keyiterator object at 0x15ca22cf0>
1
3
```

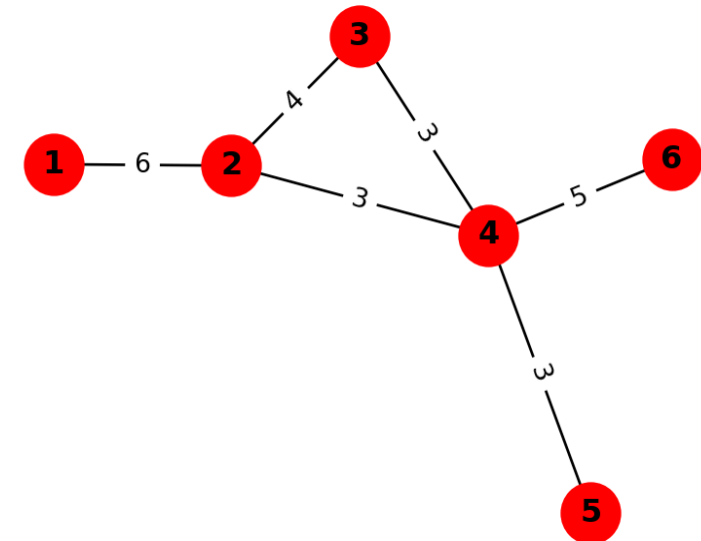


Note the edge count is 8, not 6!

Weighted networks

```
1 W = nx.Graph()
2 W.add_edge(1,2,weight=6)
3
4 W.add_weighted_edges_from([(4,5,3),(4,6,5),...])
5
6 W.edges()
7
8 W.edges(data='weight')
9
10 # Print edges with weight > 3
11 for (u,v,d) in W.edges(data='weight'):
12     if d>3:
13         print('%d, %d, %d'%(u,v,d))
14
15 # Draw the directed graph with weights
16 plt.figure(figsize=(4, 3))
17 pos = nx.spring_layout(W) # Spring layout
18 nx.draw(W, pos,
19         with_labels=True,
```

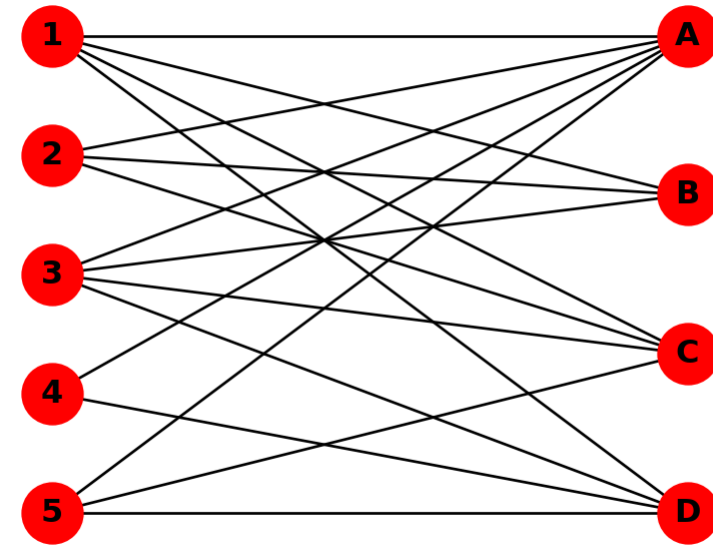
(1, 2, 6)
(2, 3, 4)
(4, 6, 5)



Bipartite networks

In a **bipartite** network, there are two groups of nodes such that links only connect nodes from different groups and not nodes from the same group

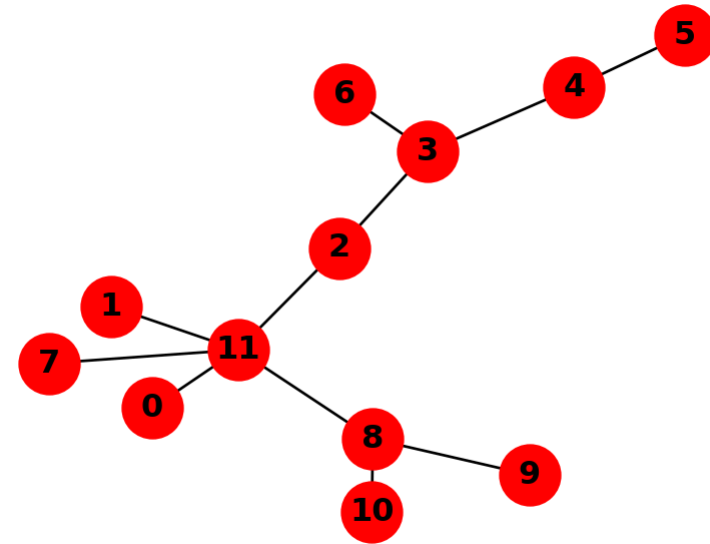
Examples of bipartite networks include those that capture the relationships between movies and stars, between songs and artists, between classes and students, and between products and customers.



Tree networks

A **tree** network is a special class of undirected, *connected* network such that the deletion of any one link will disconnect the network into two components

Examples of tree networks include phylogenetic trees, water distribution or power grids, broadcasting networks, food webs



i Note

The number of links in a tree is $L = N - 1$.

Homework

Explore some of the many **network generators** available on NetworkX. In particular, it will be helpful for forthcoming lectures / lab classes to spend some time exploring:

- **random graphs**
- **random clustered graphs**,
- **community-based graphs** (including the so-called **stochastic block model** [2]),
- **graphs with a given joint degree sequence**.

Density and sparsity

For an **undirected** network of size N (= number of nodes), with L links:

- The maximum possible number of links is: $L_{max} = \binom{N}{2} = \frac{N(N-1)}{2}$
- Its density d is given by $\frac{L}{L_{max}} = \frac{2L}{N(N-1)}$

Definition

We say the network is **sparse** if $d \ll 1$.

For a **directed** network:

- the maximum possible number of links is: $L_{max} = N(N-1)$
- The density d is then: $\frac{L}{L_{max}} = \frac{L}{N(N-1)}$

Example 1: Complete network

A **complete** network is a network in which all pairs of nodes are connected. What is its density?

```
1 CG = nx.complete_graph(8471)
2 print(nx.density(CG)) # what does this print?
```

1.0

Example 2: Facebook

Rough order-of-magnitude approximations:

- $N \approx 10^9$
- $L \approx 10^3 \times N$
- $d \approx \frac{L}{N^2} \approx \frac{10^3 N}{N^2} \approx \frac{10^3}{10^9} = 10^{-6}$

Most (but not all) real-world networks are similarly sparse because the number of links scales proportionally to N , whereas the maximum scales with N^2 .

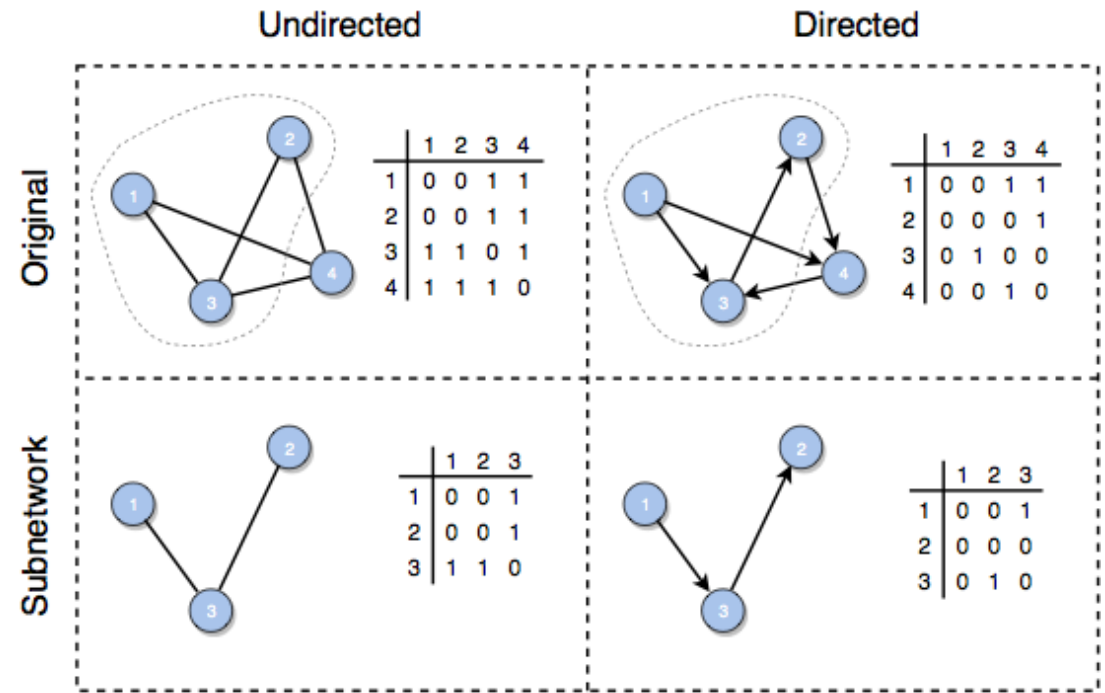
Table 1.1 Basic statistics of network examples. Network types can be (D)irected and/or (W)eighted. When there is no label the network is undirected and unweighted. For directed networks, we provide the average in-degree (which coincides with the average out-degree).

Network	Type	Nodes (N)	Links (L)	Density (d)	Average degree ($\langle k \rangle$)
Facebook Northwestern Univ.		10,567	488,337	0.009	92.4
IMDB movies and stars		563,443	921,160	0.000006	3.3
IMDB co-stars	W	252,999	1,015,187	0.00003	8.0
Twitter US politics	DW	18,470	48,365	0.0001	2.6
Enron Email	DW	87,273	321,918	0.00004	3.7
Wikipedia math	D	15,220	194,103	0.0008	12.8
Internet routers		190,914	607,610	0.00003	6.4
US air transportation		546	2,781	0.02	10.2
World air transportation		3,179	18,617	0.004	11.7
Yeast protein interactions		1,870	2,277	0.001	2.4
C. elegans brain	DW	297	2,345	0.03	7.9
Everglades ecological food web	DW	69	916	0.2	13.3

Subnetworks

A **subnetwork** is a network obtained by selecting a subset of the nodes and all of the links among these nodes.

```
1 S = nx.subgraph(G, node_list)
```



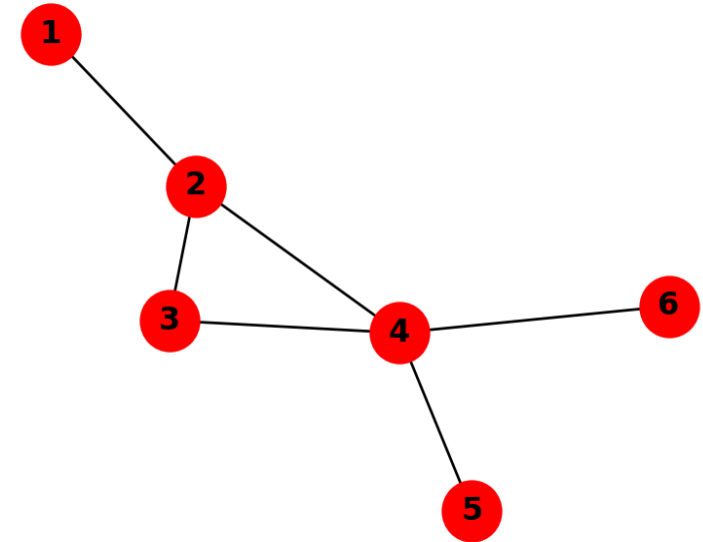
Note the matrix-like network representation!

Degree (1/2)

The **degree** of a node is its number of links, or neighbours.
The degree of node i is typically denoted by k_i .

A node without neighbours is called a **singleton** ($k = 0$).

```
1 G.degree(2) # returns the degree of node 2
2 G.degree()  # dict with the degree of all nodes
```



3

```
DegreeView({1: 1, 2: 3, 3: 2, 4: 4, 5: 1, 6: 1})
```

Degree (2/2)

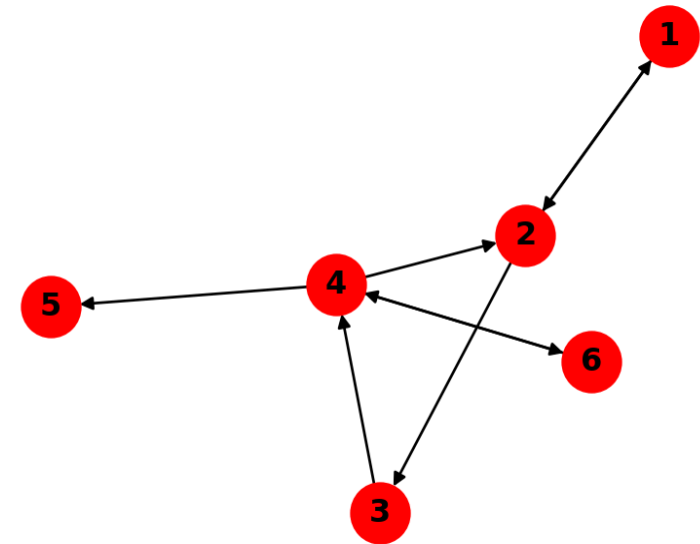
In a *directed* network, we have:

- **in-degree** of a node = number of incoming links k_i^{in}
- **out-degree** of a node = number of outgoing links k_i^{out}

```
1 D.in_degree(4)
2 D.out_degree(4)
3 D.degree(4) # total degree
```

! Total degree

The degree function returns the total degree which is the sum of both in- and out-degrees.



2
3
5

Average degree

The **average degree** of a (*undirected*) network is $\langle k \rangle = \frac{\sum_i k_i}{N}$

Note

We can connect *network size*, *number of links*, *density*, and *average degree*!

In *undirected* networks:

$$\langle k \rangle = \frac{2L}{N} = \frac{dN(N-1)}{N} = d(N-1)$$

$$d = \frac{\langle k \rangle}{N-1} = \frac{\langle k \rangle}{k_{max}}$$

Excess degree (a little bit advanced)

The **excess degree** of a node with degree k is $k - 1$. It represents the number of edges connected to that node excluding the edge that was used to reach it.

We can then define the **mean excess degree** of a network, which is the average number of connections a randomly chosen neighbour of a node has *excluding the link to the starting node*. This concept is important when considering spreading on a network (e.g., in epidemiology). The famous **Friendship paradox** [3] (not to be confused with the Friendship theorem!) is closely related to that (see Lab Class 1 and future lecture).



Tip

For more information, see this short [video](#)

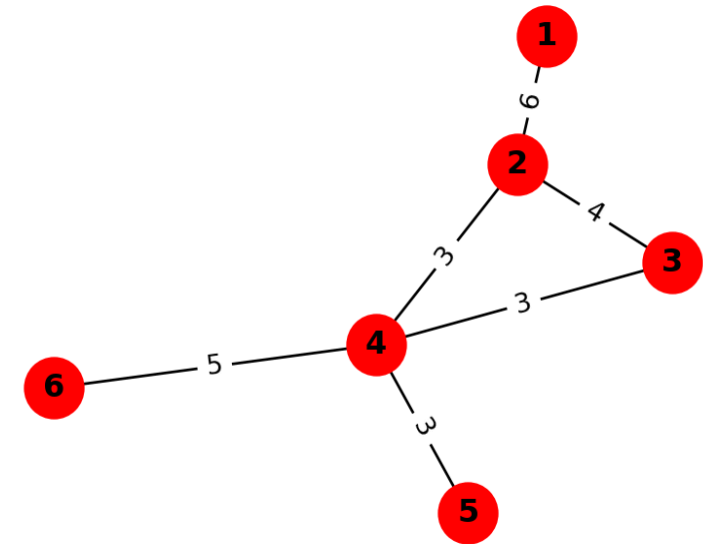
Strength

In a weighted *undirected* network, the **strength** of a node $s_i = \sum_j w_{ij}$ (a.k.a. **weighted degree**)

In a weighted *directed* network, we have:

- in-strength $s_i^{in} = \sum_j w_{ji}$
- out-strength $s_i^{out} = \sum_j w_{ij}$

```
1 W.degree(4)
2 W.degree(4, weight='weight') # strength
```

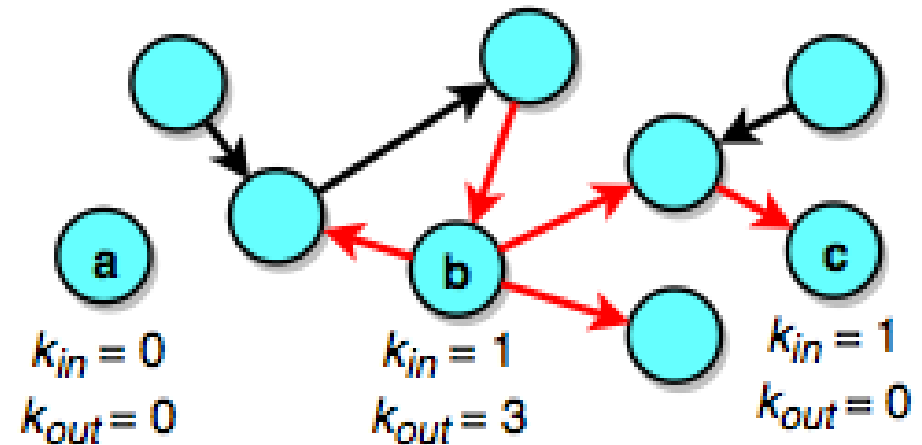
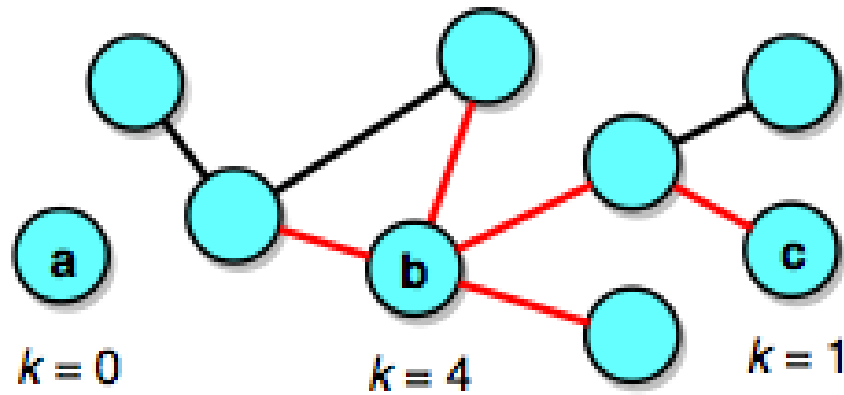


4
14

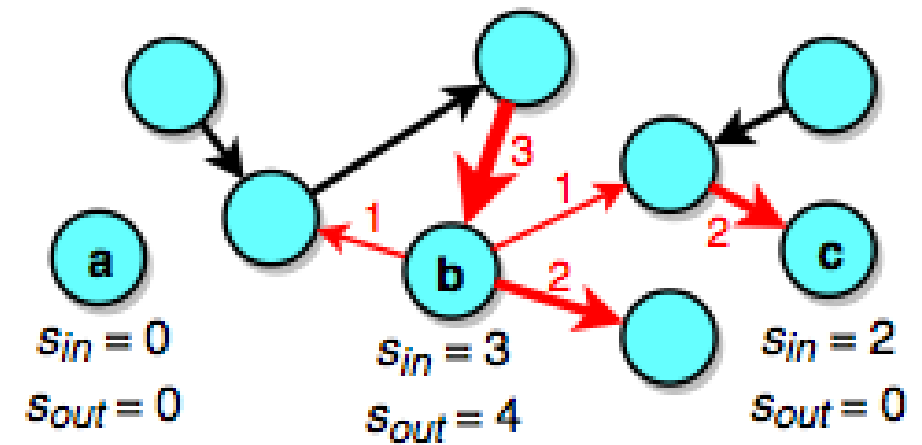
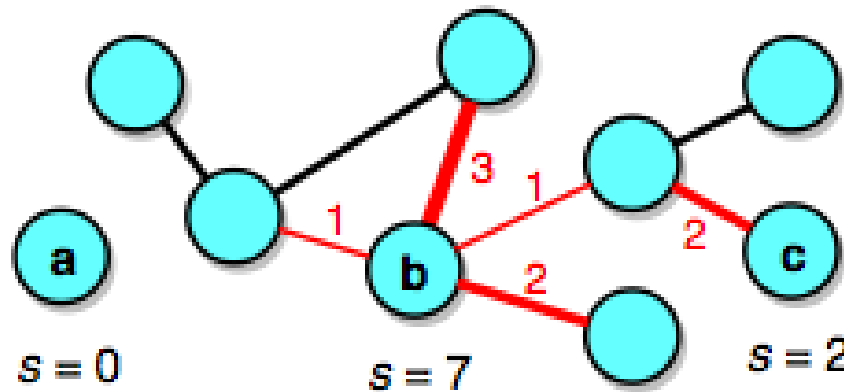
Undirected

Directed

Unweighted



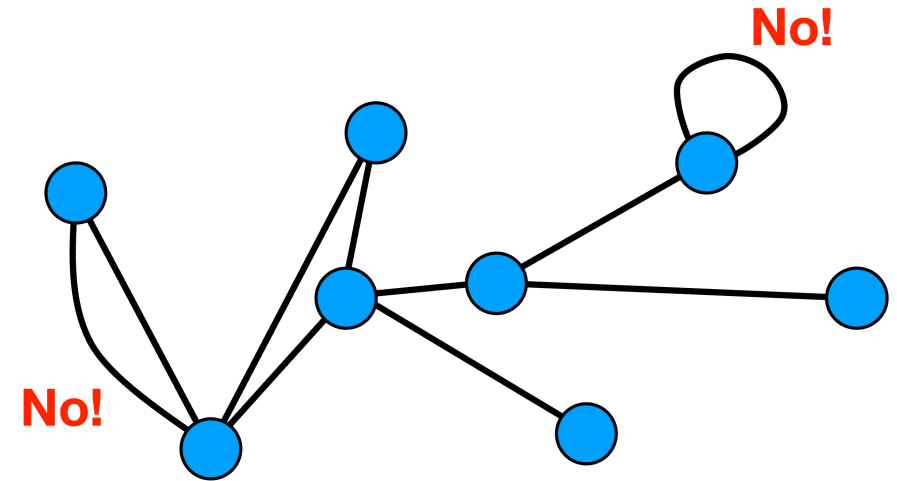
Weighted



Simplifying assumptions

We will assume:

- single-layer networks with a single type of nodes and a single type of link
- no self-loops
- at most a single link between two nodes (except for *directed* networks with links with opposite directions)



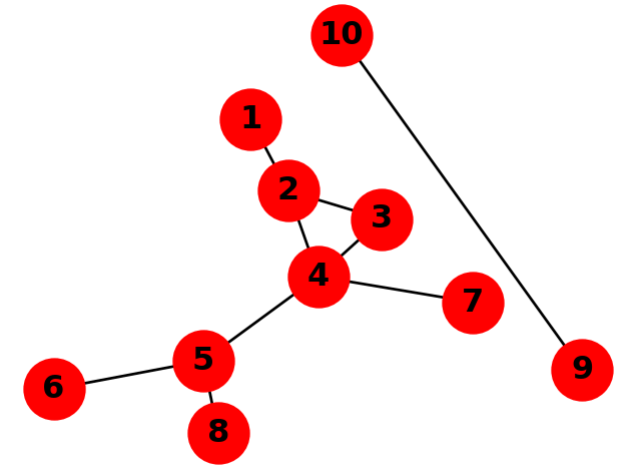
Network representations (1/4)

Adjacency matrix : $N \times N$ matrix where each element $a_{ij} = 1$ if i and j are adjacent, 0 otherwise

The diagonal elements (i.e., a_{ii}) are zero because we have no self-loops

In *undirected* networks, the matrix is symmetric: $a_{ij} = a_{ji} \quad \forall i, j$

```
1 print(nx.adjacency_matrix(G).toarray())
2 print(G[4])
3 G[3][4]['color']='blue'
4 print(G[4])
```



```
[[0 1 0 0 0 0 0 0 0 0]
 [1 0 1 1 0 0 0 0 0 0]
 [0 1 0 1 0 0 0 0 0 0]
 [0 1 1 0 1 0 1 0 0 0]
 [0 0 0 1 0 1 0 1 0 0]
 [0 0 0 0 1 0 0 0 0 0]
 [0 0 0 1 0 0 0 0 0 0]
 [0 0 0 0 1 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 1]
 [0 0 0 0 0 0 0 0 1 0]]
```

```
{2: {}, 3: {}, 5: {}, 7: {}}
{2: {}, 3: {'color': 'blue'}, 5:
 {}, 7: {}}
```

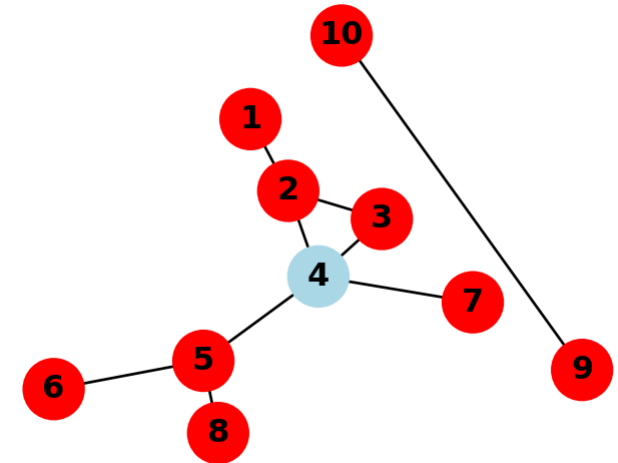
Network representations (2/4)

In *undirected* networks, the degree is obtained by summing adjacency matrix elements across rows or columns:

$$k_i = \sum_j a_{ij} = \sum_j a_{ji}$$

💡 Why is a_{ij} from i to j rather than from j to i ?

Matrix multiplication! See these [optional slides](#)



```
[ [0 1 0 0 0 0 0 0 0 0]
  [1 0 1 1 0 0 0 0 0 0]
  [0 1 0 1 0 0 0 0 0 0]
  [0 1 1 0 1 0 1 0 0 0]
  [0 0 0 1 0 1 0 1 0 0]
  [0 0 0 0 1 0 0 0 0 0]
  [0 0 0 1 0 0 0 0 0 0]
  [0 0 0 0 1 0 0 0 0 0]
  [0 0 0 0 0 0 0 0 0 1]
  [0 0 0 0 0 0 0 0 1 0] ]
```

Network representations (3/4)

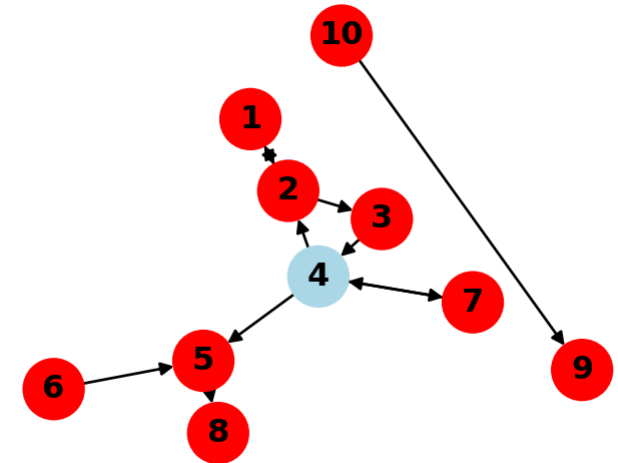
In *directed* networks, the adjacency matrix is **not** symmetric

The out-degree is obtained by summing adjacency matrix elements across rows:

$$k_i^{out} = \sum_j a_{ij}$$

The in-degree is obtained by summing adjacency matrix elements across columns:

$$k_i^{in} = \sum_j a_{ji}$$



0	1	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0
0	1	0	0	1	0	1	0	0	0
0	0	0	0	0	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	0

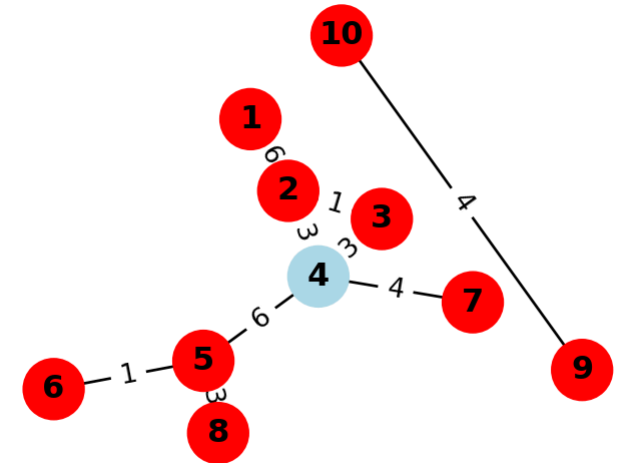
Network representations (4/4)

In *weighted* networks, element w_{ij} represents the weight of the link between i and j . It is **0** if there is no link

If *undirected*, the strength is obtained by summing adjacency matrix elements across rows **or** columns

If *directed*, the in/out-strength is obtained by summing adjacency matrix elements across columns/rows

```
1 W.degree(4, weight='weight') # strength
```



[	0	6	0	0	0	0	0	0	0	0]
[	6	0	1	3	0	0	0	0	0	0]
[	0	1	0	3	0	0	0	0	0	0]
[	0	3	3	0	6	0	4	0	0	0]
[	0	0	0	6	0	1	0	3	0	0]
[	0	0	0	0	1	0	0	0	0	0]
[	0	0	0	4	0	0	0	0	0	0]
[	0	0	0	0	3	0	0	0	0	0]
[	0	0	0	0	0	0	0	0	4	]
[	0	0	0	0	0	0	0	4	0	]

16

Quick exercise

What about multi-layer networks?

Sparse network representations

- The memory/disk storage needed by an adjacency list is proportional to N^2
- In sparse networks (most real-world networks), this is terribly inefficient: most of the space is wasted storing zeros (non-links); for very large networks, adjacency lists are unfeasible
- It is much more efficient, often necessary, to store only the actual links, and assume that if a link is not listed it means it is not present

There are two commonly used sparse networks representations:

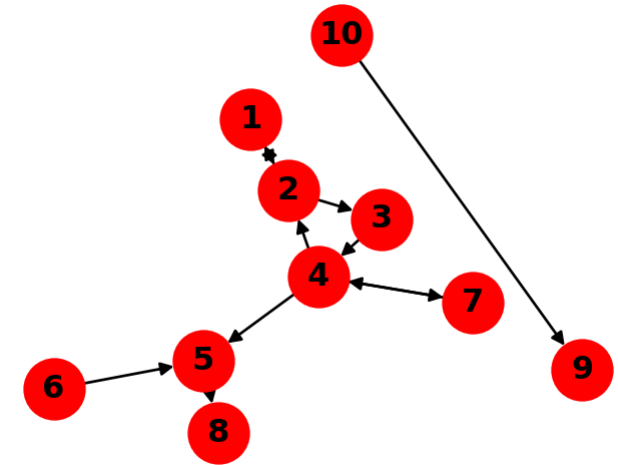
- **Adjacency list**
- **Edge list**

Adjacency list

List of neighbours for each node

```
1 for line in nx.generate_adjlist(G):
2     print(line)
3
4 nx.write_adjlist(G, "netfile.adjlist")
5 G2 = nx.read_adjlist("netfile.adjlist") \# G and G2 are
```

- In *undirected* networks, each link is listed **once** only.
- In *weighted* networks, each neighbour is replaced by a pair (neighbour, weight)



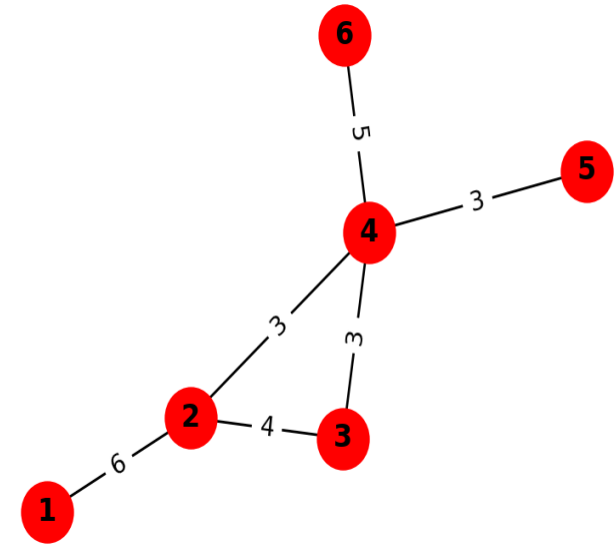
```
1 2
2 1 3
3 4
4 2 5 7
5 8
6 5
7 4
8
10 9
9
```

Edge list

List of node pairs that are connected.

In *weighted* networks, each pair is replaced by a triple (i,j weight)

```
1 # Generate and print the edge list
2 for edge in nx.generate_edgelist(W, data=["weight"]):
3     print(edge)
4
5 nx.write_edgelist(G, "netfile.edgelist")
6 G2 = nx.read_edgelist("netfile.edgelist") # G and G2 are the same
7
8 nx.write_weighted_edgelist(W, "wf.edges") # store weighted edges
9 W2 = nx.read_weighted_edgelist("wf.edges") # W and W2 are the same
```



```
1 2 6
2 3 4
2 4 3
3 4 3
4 5 3
4 6 5
```

About network drawing

- A **network layout algorithm** places nodes *on a plane* to visualize the structure of the network
- There are many layout algorithms; the most commonly used are **force-directed layout** (a.k.a. **spring layout**) algorithms. This simulates a physical system where adjacent nodes are connected by springs and otherwise repel each other:
 - Connected nodes are placed near each other
 - Links have similar length
 - Link crossings are minimized

Warning

Do not to rely too much on the layout to infer network properties!

Note

Do explore the NetworkX [documentation](#) on layouts (e.g., during the lab classes)

References

1. P. Holme & J. Saramäki, Temporal networks. *Physics Reports*, **519** (2012) 97–125. <https://doi.org/10.1016/j.physrep.2012.03.001>.
2. P. W. Holland, K. B. Laskey, & S. Leinhardt, Stochastic blockmodels: First steps. *Social Networks*, **5** (1983) 109–137. [https://doi.org/10.1016/0378-8733\(83\)90021-7](https://doi.org/10.1016/0378-8733(83)90021-7).
3. S. L. Feld, Why Your Friends Have More Friends Than You Do. *American Journal of Sociology*, **96** (1991) 1464–1477. <https://doi.org/10.1086/229693>.

About adjacency matrices and matrix multiplication (1/3)

Suppose we have 3 nodes X, Y, Z with the following *directed* edges:

- $X \rightarrow Y$
- $Y \rightarrow Z$
- $Z \rightarrow X$

The adjacency matrix is:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

What is $\mathbf{A}^2 = \mathbf{A} \cdot \mathbf{A}$?

$$\begin{aligned} \mathbf{A}^2 &= \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \end{aligned}$$

Can you see what is happening?

About adjacency matrices and matrix multiplication (2/3)

What is $\mathbf{A}^3 = \mathbf{A}^2 \cdot \mathbf{A}$?

$$\mathbf{A}^3 = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

What does it mean?

Can you see how useful this could be?

Homework

Design some simple *directed* or *undirected* networks, calculate $\mathbf{A}^3$ and build an intuition.

About adjacency matrices and matrix multiplication (3/3)

Let $\mathbf{v} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$

What is $\mathbf{u} = \mathbf{v}^T \cdot \mathbf{A}$?

How is this useful?

What is $\mathbf{u} = \mathbf{A} \cdot \mathbf{v}$?

How is this useful?

[\[back\]](#)